

SYSTEM ARCHITECTURE

Aureon

LOCAL-EMAIL-STACK

Technical Architecture Reference

June 16, 2026

Aureon / local-email-stack — System Architecture

Executive Summary

Aureon (internally `local-email-stack`) is a production cold-email-marketing platform that runs the full outbound funnel for a multi-client agency: it sources published business and creator contact emails across many public channels, enriches and verifies them, sends multi-step personalized sequences from warmed per-persona subdomains, handles inbound replies and bounces, and reports per client. It was built solo with AI assistance over roughly one month and currently runs about ten live client brands (Aureon, Diraya, AlgoAlpha, LK Advertising, ENER-G, Mercury Scales/Dorian, mark-eting, atalsolidrocks, and others) off a single shared control plane. The live runtime is a lightweight Python + Supabase + Resend stack on the operator's PC, now being productized into a self-serve client SaaS with onboarding, e-signed contracts, and a per-client dashboard.

Doc/code divergence, stated up front. The repository's top-level `README.md`, `ARCHITECTURE.md`, and `INNOVATIONS.md` describe an aspirational "v4" design (home GPU server + Oracle ARM VM + Cloudflare Email Workers, orchestrated by n8n with local Qwen LLMs, a Postal MTA, Twenty/NocoDB). That stack is **not** what runs. The authoritative description of the live system is `INSTALL.md` plus the code under `sequences/`, `scripts/`, `profiles/`, `supabase/`, `saas/`, and `desktop/`. This document treats the live Python + Supabase + Resend stack as ground truth and treats the v4 docs as vision/roadmap. Where a "v4" component is named, it is flagged as design-only.

1. System Overview

The problem it solves

A one-operator agency needs to run cold outreach for many client brands at once without a stack of paid SaaS subscriptions (lead databases, email finders, sequencers, CRMs) and without those subscriptions' shared-IP deliverability ceilings. Each client needs its own sending identity, its own warmed domains, its own branded copy and reports, and strict isolation so one client's volume or reputation never bleeds into another's. The operator needs the whole thing to run hands-free on scheduled tasks, to protect sender reputation automatically, and to keep every action inside legal/published-contact bounds.

The end-to-end pipeline in one read

A per-niche YAML config names a client (`profile_slug`) and a targeting spec. Scheduled scrapers walk that spec, extract published emails with name/company/city context, run a no-solicitation compliance gate, verify deliverability (MX-only by default, real SMTP probing staged on a port-25-open VPS), and upsert verified prospects into Supabase. An enrollment step turns a prospect plus a sequence into a `run`. The sequence runner ticks every few minutes, picks a warmed (persona, subdomain) sender under reputation/quota/timezone guards, renders branded HTML copy personalized from prospect data, sends via Resend, logs the result, and schedules the next step. Inbound replies and bounces are polled from one IMAP mailbox, matched back to the originating

send, used to pause sequences and suppress bad addresses, and drafted-and-answered by a local Claude CLI. A reporting layer renders branded HTML dashboards emailed to each client and to the operator.

2. Architecture at a Glance

Pipeline flow



Major subsystems

Subsystem	Responsibility
Data layer & schema	Supabase Postgres control plane: profiles, copy, sequences, prospects, runs, sends, replies, plus SaaS onboarding/contract/auth/CRM. RLS-scoped.
Lead sourcing & scraping	Discovers published contact emails across team pages, Google Places, German Impressum, YouTube/TradingView/social, crypto projects; compliance-gated.
Enrichment & verification	Turns raw leads into rich, personalizable, deliverable records; verifies each address exists before any send.
Sequence engine & rendering	Defines and executes multi-step cadences with A/B variants; renders branded per-client HTML; advances runs.
Sending & deliverability	Persona/subdomain rotation, warmup ramping, runtime safeguards, send windows, DMARC/SPF/DKIM provisioning, suppression.
Replies, reporting & AI	Ingests inbound mail, pauses/stops sequences, AI-drafts replies via local Claude CLI, renders branded reports.
Onboarding, profiles, compliance & legal	Turns a client spec into a provisioned identity, AI copy, Resend domains, a signed contract, an unsub page; enforces guardrails.
Client surfaces, desktop & ops	SaaS portal, Tauri desktop control panel, container/cron/Windows-task plumbing, watchdog, keep-awake, tracking worker.

3. Technology Stack

Layer	Concrete technology
Language	Python 3.12 (runtime spine), TypeScript (SaaS), Rust (Tauri desktop), JavaScript (Cloudflare Worker)
Datastore	Supabase-hosted PostgreSQL (project <code>ccmqkljsjiuavpydbkva</code>) over PostgREST; Realtime; Auth; Edge Functions. JSONB-heavy modeling.
HTTP	httpx (Python), <code>@supabase/supabase-js</code> (web), reqwest/tokio-postgres (Rust)
Sending	Resend HTTP API (<code>api.resend.com/emails</code>) per-client key; Hostinger SMTP 465 fallback; per-client Hostinger/Cloudflare/Spaceship DNS APIs
Verification	dnspython (MX/TXT), smtplib (port-25 RCPT probing, staged on a VPS), self-hosted Reacher backend
AI / LLM	Local Claude CLI (Claude Max plan, no API key) for reply and copy drafting; Anthropic API + <code>web_search</code> for metro research; Ollama (qwen2.5:32b, nomic-embed-text) via LiteLLM for the containerized sourcing/scoring services
ML	scikit-learn (LogisticRegression lead scoring), numpy, Thompson/Beta sampling bandit
Frontend (SaaS)	React 18 + Vite 5 + react-router-dom 6 + TypeScript 5
Desktop	Tauri 2 (Rust + React/Vite + Monaco + recharts)
Containers	Docker + docker compose (v4 self-hosted design and the enricher/bandit/sourcing services); python:3.12-slim VPS deploy image

Layer	Concrete technology
Scheduling	Windows Task Scheduler (LES- * tasks) on the PC; Linux cron (deploy/crontab , 1:1 mirror) on the VPS
Tracking	Cloudflare Worker (open pixel + click redirect, PATCHes send_log via service-role key)
Browser automation	Playwright / Chromium (+ playwright-stealth) for SPA/social scraping, preview screenshots, contract PDF rendering

4. Data Model

The canonical store is one Supabase Postgres project. The schema began single-tenant ("anon full access") and evolved into auth-scoped multi-client across migrations 003 through 008. A separate self-hosted Postgres (used only by the Docker enricher/bandit/sourcing services and the Tauri desktop panel) holds a different schema (leads_raw , leads_enriched , outbound_messages , bandit variants / lead_features) and is not the same database.

Core tables (schema.sql)

Table	Key columns	Relationships
profiles	PK slug ; config jsonb , active , lead_intent jsonb (004)	Parent of nearly everything
variants	PK id uuid ; subject , body , angle ; UNIQUE(profile_slug , n)	FK profile_slug -> profiles
sequences	PK id ; stop_on_reply , stop_on_bounce ; UNIQUE(profile_slug , slug)	FK profile_slug -> profiles
sequence_steps	PK id ; delay_days , inline_subject/body , forced_persona ; UNIQUE(sequence_id , step_n)	FK sequence_id -> sequences, FK variant_id -> variants
prospects	PK id ; verification + enrichment cols , unsubscribe_token UNIQUE, categorization cols (003); UNIQUE(profile_slug , email)	FK profile_slug -> profiles
runs	PK id ; status , current_step , next_send_at ; UNIQUE(sequence_id , prospect_id)	FK sequence_id , FK prospect_id
send_log	PK id ; from/to_addr , resend_id , delivered/bounced/replied/complained , opened_at/clicked_at	FK run_id -> runs
replies	PK id ; class (reply bounce complaint unrelated) , raw_headers jsonb	nullable FK run_id , FK profile_slug
warmup_state	PK profile_slug ; enabled , current_day , reputation jsonb	FK profile_slug -> profiles

Key indexes that shape runtime behavior: idx_runs_next_send_at is a partial index WHERE status='queued' that the runner uses to find due work; prospects carries indexes on verified , niche ,

enriched_at, unsubscribe_token, plus quality/audience and a GIN index on industry_tags[].

Migration additions

Migration	Adds	Purpose
003 scale_pipeline	prospect_candidates (discovery queue), prospect categorization cols (source_platform, audience_size, industry_tags[], geo, quality_score)	Replaces flat .txt queue files with a Supabase work queue
004 lead_intent	profiles.lead_intent jsonb, search_jobs queue	Per-client targeting spec + "go find more leads" requests
005 onboarding	clients, onboarding_submissions, provisioning_status	SaaS onboarding work queue
006 contracts	contracts (draft -> signed -> sealed), UNIQUE(submission_id)	Self-hosted e-sign; provisioning gated on a signed contract
007 auth	user_roles, is_admin() / owns_slug() SECURITY DEFINER helpers, per-owner policies	Replaces anon-full-access with auth-scoped RLS
008 crm	invoices, sales	Client dashboard CRM (client-reads-own / admin-writes)

Queue and SaaS tables

Table	Key columns	Notes
prospect_candidates (003)	UNIQUE(profile_slug, source, handle); status pending claimed done failed, claimed_by/at, attempts, meta jsonb	Multi-worker safe; partial pending/claimed indexes
search_jobs (004)	profile_slug, intent_snap jsonb, status, result jsonb	Portal "Start lead search" jobs
clients (005)	auth_user_id, email, FK profile_slug ON DELETE SET NULL; UNIQUE lower(email) (007)	Links auth user to client + profile
onboarding_submissions (005)	raw_answers jsonb, status machine pending -> provisioning -> needs_dns ready -> live error	Drives onboard-pipeline.py
provisioning_status (005)	step, state, payload jsonb; UNIQUE(submission_id, step)	Per-step progress surfaced to the portal
contracts (006)	status draft signed sealed void, signer fields, contract_sha256, sealed_at, signed_pdf_path	One live contract per submission
user_roles (007)	PK user_id -> auth.users; is_admin boolean	Read-own-role SELECT policy only
invoices (008)	UNIQUE(invoice_ref); amount_cents / due_cents bigint, source manual facturx	

Table	Key columns	Notes
sales (008)	amount_cents, status won pipeline lost, closed_at	

RLS posture and how it evolved

RLS evolution is the central data-layer story. `schema.sql` plus migrations 005/006 grant blanket "anon full access" — a single-tenant model where the project URL plus anon key is the entire auth boundary. Migration 007 rips that out: it introduces `user_roles` and the SECURITY DEFINER helpers `is_admin()` and `owns_slug()`, adds per-owner SELECT/UPDATE policies on `clients`, `onboarding_submissions`, `contracts`, `sequences`, `replies`, and `runs`, and seeds the operator as admin. The public onboarding form then moves behind a service-role Edge Function (`auth-admin`), so anon loses all direct access to onboarding/contract tables.

`prospects` has its own separate RLS history in `docs/SUPABASE_RLS_FIX.sql` and `_v2`. v1 (2026-06-10) locks down prospects to anon INSERT-only to stop a PII read leak, but that would silently break every unsubscribe link because unsubscribe is an UPDATE. v2 (2026-06-12) keeps anon INSERT and adds a token-scoped anon UPDATE (`USING unsubscribe_token IS NOT NULL, WITH CHECK unsubscribed=true`) so unsubscribe keeps working with no SELECT/DELETE for anon. Migration 007 deliberately leaves `prospects` untouched.

Operationally: backend scripts must use `SUPABASE_SERVICE_KEY` (bypasses RLS); public pages use `SUPABASE_ANON_KEY` constrained by the token-scoped prospects policies. A documented gap: both `queue_lib.py` and `supabase_sync.py` as written load only the anon key from `sequences/supabase.env`, and roughly 87 anon-key call sites exist; the recommended pattern is `env.get('SUPABASE_SERVICE_KEY')` or `env['SUPABASE_ANON_KEY']` so server scripts auto-prefer the service key.

Live scale (from `_schema.json` export)

The export enumerates the 8 original core tables only: 10 profiles, 3182 prospects, 1828 runs, 3773 send_log rows, 796 replies, 81 variants, 9 sequences, 61 sequence_steps. The 003-008 additions are not in the export, indicating the export script enumerates a fixed table list.

5. Subsystem Architecture

5.1 Data layer & schema

Purpose. The cloud control plane and canonical state store. A single Supabase Postgres project holds every profile, copy variant, sequence, prospect, run, send, reply, and the SaaS onboarding/contract/auth/CRM data.

Module	Role
<code>supabase/schema.sql</code>	Base DDL: 9 core tables, indexes, realtime publication, anon-full-access RLS, <code>set_updated_at()</code> trigger
<code>supabase/migration_003..008</code>	Scale queue, lead intent, onboarding, contracts, auth-scoped RLS, CRM
<code>docs/SUPABASE_RLS_FIX.sql</code> / <code>_v2.sql</code>	prospects lockdown: anon INSERT-only, then token-scoped anon UPDATE for unsubscribe

Module	Role
<code>sequences/supabase_sync.py</code>	push/pull/status CLI: upserts local profiles/variants/warmup jsonl into Supabase; pulls profiles back as backup
<code>sequences/queue_lib.py</code>	Supabase-backed <code>prospect_candidates</code> queue: <code>enqueue</code> , atomic <code>claim_batch</code> , <code>mark_done</code> / <code>release_to_pending</code> , stats
<code>supabase/config.toml</code>	Supabase CLI config: project id + <code>auth-admin</code> Edge Function with <code>verify_jwt=false</code>

Data flow. Profiles, copy, and historical sends originate as local files and are upserted by `supabase_sync.py push` using PostgREST with `Prefer: resolution=merge-duplicates` and `on_conflict targets`. After the first push the cloud is canonical and both PCs read from it. Lead discovery flows through `prospect_candidates: harvesters enqueue()` (ignore-duplicates against the UNIQUE constraint); workers `claim_batch()` (`SELECT N ids status=pending OR claimed-and-stale`, then PATCH to claimed with `claimed_by=hostname:pid`), process into `prospects`, then `mark_done`. A prospect plus a sequence yields a `run`; the runner picks queued runs, sends, and writes `send_log`; replies/bounces land in `replies`.

External deps. Supabase (Postgres + PostgREST + Realtime + Auth + Edge Functions), httpx, `gen_random_uuid()` / pgcrypto, Resend (upstream of send results).

Notable decisions.

- The atomic claim has a documented race: PostgREST will not honor LIMIT on PATCH, so `claim_batch` does SELECT-then-PATCH and two workers can briefly grab the same ids. Accepted because the prospects upsert dedups on email; true serializability would need a Postgres RPC with `FOR UPDATE SKIP LOCKED`.
- `send_log` historical push from `.resend.jsonl` has no natural unique key and is a plain INSERT, so re-running push can create duplicate rows (noted as acceptable).
- `push_variants` intentionally requires an explicit `profile_slug` in each `variants.json` (the old directory-name fallback was dropped) to stop a stale experimental directory from overwriting a live client's canonical variants.
- An `unsubscribe_by_token` RPC is referenced in 007's header comment as the narrow anon path for prospects, but its definition is not present in any reviewed SQL file (applied directly in the dashboard).

5.2 Lead sourcing & scraping

Purpose. Discover PUBLISHED business/creator contact emails across many channels, verify each, and upsert on-ICP prospects per client. Targeting is driven entirely by per-niche YAML, so a new vertical is a config edit, not new code. The explicit stance (`docs/LEGAL_LEAD_SOURCING_UPGRADE.md`) is legal/published-contact sourcing only, gated by a no-solicitation compliance check, never private/credentialed data.

Module	Role
<code>lead_scrape.py</code>	Core team-page scraper + shared library (httpx -> Playwright fallback, 3-pass email extraction, quality+compliance gates). Exports <code>ScrapedLead</code> , <code>fetch_html</code> , <code>extract_leads_from_page</code> , <code>supa_upsert_prospect</code> , <code>load_niche</code>

Module	Role
<code>places_scrape.py</code>	Google Maps/Places via Serper <code>/places</code> ; same-registrable-domain email rule; workhorse for the 6 local-business brands
<code>seed_discover.py</code>	Auto-grows niche <code>seeds</code> : lists via a Serper -> Brave -> keyless-scraper -> ddgs backend chain
<code>impressum_scrape.py</code>	DACH source: legally-mandated Impressum page -> email + Geschaeftsfuehrer + company + PLZ city; German anti-Werbung gate
<code>auto_source_atal.py</code>	Daily autonomous domain discovery for atalsolidrocks; hands new domains to <code>impressum_scrape</code>
<code>youtube_scraper.py</code> / <code>youtube_worker.py</code>	YouTube Data API v3 (sync scraper with fan-out to IG/X/TikTok queues; async sem=10 worker off the candidate queue)
<code>tradingview_scrape.py</code>	AlgoAlpha-specific: paginates <code>/scripts</code> , resolves author site via <code>?utm_source=tradingview</code> tag
<code>social_scrape.py</code>	Instagram/X/TikTok (Playwright + regex/deobfuscation) and Twitch (Helix); consumes curated handle files
<code>crypto_projects_scrape.py</code>	DefiLlama + CoinMarketCap project sites; owns <code>_is_junk_email</code> reused across scrapers
<code>search_dispatch_worker.py</code>	Bridges the portal's "Start lead search" button to sourcing via <code>search_jobs</code> + <code>lead_intent</code>
<code>name_derive.py</code>	<code>derive_first_name</code> / <code>derive_company</code> / <code>is_free_or_isp_domain</code> quality helpers
<code>niches/*.yaml</code>	~21 per-vertical targeting configs

Data flow. A niche YAML names a `profile_slug` and either seed URLs, a `places` block, or a handle/domain file. A scheduled driver runs the matching scraper. Each candidate passes a compliance gate (`compliance.forbids_outreach` aborts on any EN no-solicitation or German anti-Werbung notice), then a 3-pass extractor (script-tag JSON pairs, `mailto:` , plaintext) mines the surrounding card for name/title/phone. A quality gate filters junk/role local-parts and blocklisted domains, applies structural false-match guards (Sentry DSNs, version strings, file-extension TLDs), derives missing `first_name`/company, and hard-rejects if `require_first_name` (per niche) or company is missing. Survivors pass through `lead_verify.verify` (MX-only by default) and upsert into `prospects` on conflict (`profile_slug` , `email`) with verification metadata and `enriched_context` . Cursor files (`.seeds.done` , `.places.done` , `.<platform>.done`) make runs incremental and idempotent.

External deps. Supabase PostgREST; Serper.dev `/search` and `/places` ; Brave Search (fallback); keyless DDG/Startpage/Mojeek + ddgs; YouTube Data API v3; Twitch Helix; DefiLlama; CoinMarketCap; Playwright/Chromium; httpx, BeautifulSoup(lxml), PyYAML. Google CSE is present in code but dead (closed to new customers ~Jan 2026; kept only as a grandfather fallback).

Notable decisions.

- TWO parallel sourcing systems exist. The LIVE system is the `sequences/*.py` scripts writing to Supabase `prospects` . `docker/sourcing/` is a separate "v4 universal sourcing" FastAPI design writing to its own Postgres `leads_raw` with a DIFFERENT YAML schema

(`sourcing_engines / enrichment / icp_filter`); it is not wired into the live pipeline. Of its 12 engines most return `[]` stubs; `github_developer` and `twitter_profile` are the most implemented.

- Compliance is a hard gate, not advisory, applied in `lead_scrape`, `places_scrape`, and `impressum_scrape`.
- `places_scrape` only accepts an email on the business's own registrable domain, blocking third-party/vendor/spam addresses found on the page.
- Verification is MX-only by default because port 25 is blocked on the operator's boxes; SMTP RCPT/catch-all probing is opt-in.
- Scrapers self-limit to ~480-500s to exit cleanly before a 600s orchestrator kill; progress is real because it is cursor-based and idempotent.
- `youtube_scraper` fans out IG/X/TikTok handles found in a channel About into those scrapers' queues, with an explicit guard so crypto handles never leak into the dorian niche (a real 2026-06-10 incident).

5.3 Enrichment & verification

Purpose. Turn raw leads into rich, personalizable, deliverable records, and verify that each address exists before any send so campaigns hit real inboxes and protect sender domains from bounces.

Module	Role
<code>context_enrich.py</code>	No-LLM, no-API business-context extractor: homepage + one subpage -> product/pricing/user-count/outcomes/socials into an <code>EnrichedContext</code> dataclass
<code>context_autofill.py</code>	Scheduled Supabase worker: picks eligible prospects, derives a URL (skipping free-mail), runs <code>context_enrich</code> , PATCHes <code>enriched_context</code> + <code>enriched_at</code>
<code>enrichment_worker.py</code>	Backfills <code>audience_size</code> (YouTube API), <code>industry_tags</code> , <code>geo</code> (TLD), composite <code>quality_score</code> 0-100
<code>personal_hook_worker.py</code>	Fills <code>{personal_hook}</code> for YouTube prospects from the latest video title, sanitized (no apostrophes/dashes)
<code>name_derive.py</code>	High-precision <code>first_name</code> / <code>company</code> derivation; shares <code>GENERIC_LOCAL_PARTS</code> with <code>lead_verify</code>
<code>prospect_timezone.py</code>	<code>resolve_timezone()</code> IANA tz: override -> city table -> TLD -> profile default -> UTC
<code>lead_verify.py</code>	Central verifier: syntax -> junk-local -> disposable -> placeholder -> MX -> paced SMTP RCPT -> catch-all; returns a <code>VerificationResult</code> method
<code>scripts/smtp-pool-verify.py</code>	Daily LES gate calling <code>verify(do_smtp_probe=True)</code> , hard-suppressing only definitively-dead mailboxes (550/551/553)
<code>scripts/reacher_verify.py</code>	Client for a self-hosted Reacher backend on a port-25-open VPS
<code>sequences/reverify-unknowns.py</code>	Scheduled paced SMTP re-probes of <code>smtp_unknown</code> prospects; suppresses only on a definitive 550
<code>docker/enricher/*</code>	FastAPI deep-enrichment service (website/social/infra/techstack/news/business/signals) -> <code>leads_enriched.profile</code> jsonb (separate Postgres data plane)

Module	Role
<code>docker/bandit-scorer/bandit_scorer.py</code>	Thompson-sampling variant bandit + embedding-based P(reply) lead scoring

Data flow. Two enrichment worlds feed one verification spine. The live op uses the lightweight Supabase path: `context_autofill` -> `context_enrich`, then `enrichment_worker` categorization, `personal_hook_worker`, `name_derive`, and `prospect_timezone`. The heavy Docker `enricher` runs the website -> social -> techstack -> infra -> news/business -> signals pipeline into `leads_enriched.profile` (a separate Postgres world). Verification is `lead_verify.verify()`: scrapers run MX-only, `smtp-pool-verify` runs the real RCPT probe daily on a VPS and suppresses only definitive 550/551/553, `reverify-unknowns` paces re-probes of `smtp_unknown` rows.

External deps. scraper service (`/scrape/url`), SearXNG, rdap.org, archive.org, OpenCorporates, Wikidata, GitHub/Bluesky APIs, YouTube Data API v3, Ollama (nomic-embed-text, qwen2.5), Supabase REST, self-hosted Reacher, dnspython + smtplib; fastapi/uvicorn/httpx/asyncpg/numpy/scikit-learn.

Notable decisions.

- `lead_verify.py` is the single source of truth; `name_derive` imports its `GENERIC_LOCAL_PARTS` so "not a human name" is defined once. The set is hand-curated from real bounce incidents.
- Stated principle: certainty drops a lead, uncertainty never does. `smtp_unknown` stays `verified=True` /send-eligible and is only suppressed later on a definitive 550, so a rate-limited probe cannot shrink the pool.
- Catch-all domains are deliberately marked `verified=False` (method `catch_all`) because no verifier can prove a mailbox behind an accept-all server; `reacher_verify` can optionally keep risky catch-all.
- SMTP probing is paced per MX-provider zone (`_MIN_HOST_GAP_S = 20s` + jitter) to stay under big-provider throttle thresholds; provider-format rules (e.g. Gmail username rules) reject impossible mailboxes with zero network calls.
- `personal_hook_worker._sanitize_title` strips apostrophes and em/en/word-internal dashes, enforcing the no-em-dash outbound rule at the data layer.
- The bandit lead model cold-starts at a no-op 0.5 until ≥ 50 labeled rows exist; `/variant/curate` uses qwen via Ollama to generate fresh variants from top performers and disables provably-bad arms by Beta CI upper bound.
- The Docker enricher and the live Supabase prospects path are two largely separate data planes sharing one conceptual goal.

5.4 Sequence engine & email rendering

Purpose. The core campaign engine: define per-client multi-step cadences with A/B variants, schedule and execute each step on a cron tick, pick a warmed (persona, subdomain) sender under guards, personalize copy from prospect data, render branded HTML, send via Resend, log, and advance.

Module	Role
<code>sequence-runner.py</code>	The brain. <code>tick()</code> pulls due runs, resolves step+variant (A/B split), picks/sticks a sender, runs merge + safeguard gates, sends, logs, advances. Also enqueue / enqueue-niche / status CLIs
<code>email_render.py</code>	Single source of truth for the wire format. <code>build_payload()</code> assembles Resend JSON; <code>render_html()</code> dispatches to a per-client template or generic fallback; injects self-hosted open/click tracking
<code>profile_lib.py</code>	Loads <code>profiles/<slug>.json</code> + <code>.private.json</code> overlay, synthesizes the subdomain pool, computes warmup-day ceilings, reputation checks, <code>materialize_persona</code>
<code>email_template_aureon/diraya/energ.py</code> (+ <code>algoalpha</code> , <code>lk</code> , <code>f2</code> , <code>atalsolidrocks</code> , <code>mark-eting</code>)	Per-client branded HTML templates; CTA shown only on persuasion steps 2-5
<code>sequences/<slug>-default/variants.json</code>	Per-client copy source of truth: array of {n, delay_days, angle, subject, body}
<code>scripts/wire-sequence-steps.py</code>	Idempotently upserts <code>sequence_steps</code> linking <code>step_n</code> -> <code>variant_id</code> with <code>delay_days</code>
<code>scripts/render-all-steps.py</code>	CI/QA gate: renders every (profile x step) against a real prospect, asserts no unresolved placeholders/empty greetings; exits non-zero on any artifact
<code>render_email_preview.py</code> / <code>render_previews.py</code>	Per-variant Playwright screenshot; whole-niche inbox-card preview HTML
<code>send-sequence.py</code>	Test-only direct-to-MX SMTP path (port 25 blocked in production)

Data flow. Copy authoring starts in `variants.json`; a push loads each into the `variants` table; `wire-sequence-steps.py` upserts `sequence_steps`. At runtime `tick()` (every ~5 min) calls `fetch_due_runs()` for `status='queued'` and `next_send_at<=now`, batch-prefetches `sequences/steps(+variants)/prospects/first send_log/config/today's send_log`, and per run resolves the step's copy (a step carrying BOTH inline copy and a linked variant is an A/B test, split by `md5(prospect_id)%2`). Sender selection: step 1 picks the (persona, domain) with most remaining quota via `pick_persona_and_domain`; step 2+ reuses the run's sticky sender and SKIPS the tick if that subdomain is capped. Gates run in order (verified, unsubscribed, required-merge-field presence, then `safeguards.check_all`). Merge tags render once, then `build_payload` -> `render_html` (brand template) -> `_inject_tracking` -> POST to Resend with the per-client key. `log_send` writes `send_log`; `advance_run` completes or schedules the next step at `now + delay_days` (+/-2h jitter).

External deps. Supabase PostgREST; Resend; httpx; Playwright (previews); PyYAML; self-hosted open/click tracker; the `safeguards` and `algoalpha_offer` modules.

Notable decisions.

- A/B testing is implicit and deterministic (`md5(prospect_id)%2`), so each lead always gets the same side; differing subjects let `send_log` measure reply rates.
- Sender stickiness: a lead is worked end-to-end by the SAME (persona, subdomain) pair; mid-thread drift would break the inbox thread.
- Strict no-fallback personalization: any missing REQUIRED merge field (first_name/company/personal_hook) CANCELS the run; optional/derived fields always resolve to

at least an empty string.

- Step 1 is a plain personal email with no CTA (cold-open hygiene); the branded CTA appears only on steps 2-5; value/breakup steps (6,7) stay clean.
- Self-hosted tracking is injected by `email_render` because Resend sets the `open_tracking` flag but does not actually inject the pixel/rewrite links (verified empirically).
- Per-tick hardening: batch-prefetch, run shuffle so one brand cannot starve others, and a ~13-min soft budget under the 15-min task limit (cut ~900s ticks to ~11s).
- Reply-To is always `[client mailbox, info@aureonglobal.de]` so every reply reaches both client and agency.

5.5 Sending & deliverability

See Section 8 for the deep treatment. In brief, this subsystem is the multi-channel outbound path plus reputation protection: persona/subdomain rotation, route picking, warmup ramping, deliverability scoring, runtime safeguards/send-window gating, DMARC/SPF/DKIM and subdomain auto-provisioning, and bounce suppression.

Module	Role
<code>resend-pool-send.py</code>	Primary live send path: least-loaded (persona, verified-subdomain) pick, <code>materialize_persona</code> bind, POST Resend, record jsonl + <code>send_log</code>
<code>profile_lib.py</code>	Profile+secret merge, from_domains pool, calendar-derived warmup day + snowball ceiling, reputation gate, 1:1 binding
<code>safeguards.py</code>	Six per-send guards: no-solicitation, recipient-local send window, rolling-72h reputation, global daily cap, rate limit, recipient+step dedup
<code>warmup-scheduler.py</code>	Daily per-subdomain snowball ramp; auto-pause on reputation
<code>provision_subdomain.py</code> / <code>domain_autoprovision.py</code>	Resend create-domain -> Hostinger DNS push (DKIM/SPF/DMARC) -> verify -> stamp <code>verified_at</code>
<code>deliverability-score.py</code>	Preflight 0-100 scorecard via live DNS + warmup maturity + reputation
<code>resend-status-reconcile.py</code> / <code>resend-webhook.py</code>	Hourly poller (and Svix webhook, not reliably deployed) patching <code>send_log</code> , suppressing, refreshing reputation
<code>scripts/suppress-bounced.py</code>	Cross-profile: <code>send_log.bounced</code> -> prospects <code>verified=false</code> + <code>unsubscribed=true</code>
<code>scripts/fix-dmarc.py</code>	Multi-provider DMARC repair (Cloudflare/Spaceship/Hostinger)
<code>docker/route-picker/route_picker.py</code>	FastAPI route picker (Postal-era design, not live)

5.6 Replies, reporting & AI

Purpose. Close the outbound loop: ingest inbound mail, reconcile against `send_log` to pause sequences and stop on reply, draft on-voice responses with the local Claude CLI, and render branded reports.

Module	Role
<code>imap-poll.py</code>	IMAP poller over Hostinger; classifies reply/bounce/complaint/unrelated/self_alert, matches to send_log, inserts <code>replies</code> , pauses runs, honors opt-outs, emails <code>[REPLY ALERT]</code>
<code>reply-autodraft.py</code>	AI responder: gates suppression + active-prospect, routes positive call-intent to a Calendly auto-reply, drafts a Hormozi-voice reply via the local Claude CLI, AUTO-SENDS threaded (BCC info@) or falls back to an operator draft
<code>seller-outreach.py</code>	Research-backed follow-ups: scrapes the prospect site, drafts a Calendly-driving follow-up, auto-sends, emails the operator a digest, tracks <code>custom_fields.seller_outreach</code>
<code>meeting-followup.py</code>	Post-Calendly loop: scans "New Event" emails, pops a PowerShell GUI for outcome, queues (never auto-sends) a Claude follow-up
<code>notify_daemon.py</code>	System-tray daemon (pystray + winotify): toasts for new replies/bounces/complaints/unsubscribes every 30s
<code>scripts/daily-report.py</code>	Core reporting engine: full internal ops report (KPIs, reply-intent classifier, per-step funnel, subdomain health, pipeline) via Resend
<code>scripts/client-reports.py</code> / <code>dorian-report.py</code> / <code>volume-report.py</code>	Per-client branded reports, bespoke Mercury Scales report, compact two-profile volume check
<code>scripts/daily-status.py</code> / <code>_ops-live-stats.py</code>	Console-only health snapshots
<code>scripts/research-dispatcher.py</code>	Autonomous attorney-list builder via Anthropic API + <code>web_search</code> , or a \$0 web-search heuristic, with an independent QC gate
<code>scripts/_reply-correlation-analysis.py</code>	Offline Pearson/Spearman/ML/random-forest correlation of copy features vs reply rate
<code>docker/litellm/config.yml</code>	LiteLLM gateway aliasing OpenAI/Anthropic model names onto local Ollama models (separate from the reply-drafting Claude CLI)

Data flow. Inbound mail lands in the single Hostinger mailbox `info@aureonglobal.de` (every cold send sets Reply-To there). `imap-poll.py` scans INBOX + INBOX.Junk over the last 14 days, classifies, dedupes via `replies.raw_headers` Message-ID, resolves the originating send via In-Reply-To/References with a recipient+subject fallback (Resend rewrites Message-IDs), inserts a `replies` row, marks the matched `send_log` row, and on a real reply pauses EVERY queued run for that prospect's email (`runs.status=paused_replied`), honoring opt-out text. `reply-autodraft.py` later reads `class=reply` rows with `autodraft_sent` unset, applies hard suppression + active-prospect gating, resolves brand+persona from `run_id`, and either sends a Calendly auto-reply for positive intent or drafts via the local Claude CLI and AUTO-SENDS threaded from the original sender (BCC info@). The reporting layer reads `send_log/replies/prospects/runs/profiles/sequences`, attributes by sending subdomain, and emails branded HTML.

External deps. Hostinger IMAP/SMTP; Resend; Supabase REST; local Claude CLI (`claude.cmd`, Max plan, no API key); Anthropic API + `web_search`; Calendly (parsed emails); LiteLLM + Ollama; `httpx/pystray/Pillow/winotify`; `numpy/pandas/scipy/scikit-learn/matplotlib`; PowerShell WinForms.

Notable decisions.

- Reply-stop is intentionally `run_id`-independent: because Resend rewrites Message-IDs, `imap-poll` pauses every queued run for the sender's email rather than relying on header matching.
- A safety gate: a threaded "reply" matching no send from an unknown address is downgraded to "unrelated"; a non-threaded message from a known prospect is upgraded to "reply".
- A hard suppression list (`laso.finance` is an active legal case, plus own-brand domains) is enforced in both `imap-poll` and the draft scripts so those senders are never alerted, paused, or auto-answered.
- The Claude CLI is invoked with a full `--system-prompt` replacement, all tools disabled, `--setting-sources user`, and an empty temp cwd, deliberately so the agent will not inspect the workspace and refuse on cold-email grounds; outputs are dash-scrubbed and preamble-stripped.
- Auto-send mode (set 2026-06-11) answers genuine campaign replies automatically; only `meeting-followup` stays approve-first.
- Report attribution is by sending subdomain (collision-free), explicitly NOT by `persona_slug` (persona first-names collide across brands and previously dumped Aureon volume onto atalsolidrocks).

5.7 Onboarding, profiles, compliance & legal

Purpose. Turn a single client spec (a SaaS form submission or an operator-written profile) into a fully provisioned identity: a per-client profile, an AI-drafted 7-email sequence, created Resend domains, a digitally signed pilot agreement, a branded unsubscribe page, and the compliance guardrails.

Governing principle: a new client is config, not custom code.

Module	Role
<code>onboard-pipeline.py</code>	SaaS auto-kickoff: consumes pending submissions, gates on a sealed contract, builds profile -> AI-drafts copy -> provisions domains -> queues leads -> stages warmup, writing progress to <code>provisioning_status</code>
<code>contract-sign.py</code>	Self-hosted e-sign backend (prepare/seal/run): auto-prepares a draft, on signature stamps the audit trail (IP, UA, SHA-256), renders a locked PDF via Playwright, flips to sealed, emails dual evidence copies
<code>contract_lib.py</code>	Maps <code>raw_answers</code> into the pilot-agreement HTML by swapping client strings into the Diraya base template; <code>verify_clean()</code> blocks base-template leaks and em dashes
<code>compliance.py</code>	<code>forbids_outreach(text)</code> : EN no-solicitation + German Impressum anti-Werbung detector, used at scrape time
<code>safeguards.py</code>	Runtime send-time guards (<code>check_all</code>); first failure aborts the send
<code>brand_extract.py</code> / <code>brand_autofill.py</code>	Visual brand miner + scheduled autofiller that MERGES results preserving operator-owned keys
<code>scripts/site_style.py</code>	Browser-free site-style extractor with full profile-brand fallback
<code>scripts/build-unsub-pages.py</code>	Brand-matched GitHub-Pages unsubscribe page per profile, calling the <code>unsubscribe_by_token</code> RPC

Module	Role
<code>scripts/scaffold-niche.py</code>	Turns a client ICP into <code>niches/<slug>.yaml</code> + a <code>PROFILE_CFG</code> snippet
<code>profiles/<slug>.json</code>	The per-client data model (brand, relay/from_domains, personas, rotation, warmup curve, send_window); secrets in <code>.private.json</code>

Data flow. SaaS path: the public app writes an `onboarding_submissions` row (status=pending). `contract-sign.py prepare()` generates a draft contract; the client signs in the browser (status=signed); `seal()` SHA-256-hashes the agreed bytes, builds the e-signature certificate panel, renders a locked PDF, flips to sealed, and emails dual evidence copies. `onboard-pipeline.py process()` is hard-gated on `sealed_contract()` — with no sealed contract it parks the submission at `awaiting_signature`. Once sealed it slugifies the company, `build_profile()` assembles the profile (4 starter subdomains, 4 personas each 1:1-bound, snowball curve, weekday 08-17 window), `push_profile_db()` upserts with the Resend key stripped, `draft_sequence()` shells the local Claude CLI for a 7-variant sequence (em-dash scrubbed), and `provision_domains()` creates subdomains in Resend and hands DNS records back as a `needs_input` step. Independently, `brand_autofill` merges mined brand into profiles, and `build-unsub-pages.py` renders the per-profile unsub page. At send time `safeguards.check_all()` enforces the no-solicitation backstop first.

External deps. Supabase PostgREST; Resend (domains + alert egress); local Claude CLI; Hostinger SMTP (evidence emails); Playwright/chromium (PDF); httpx; BeautifulSoup/lxml; GitHub Pages; zoneinfo.

Notable decisions.

- Hard contract gate: no provisioning of any kind until `sealed_contract()` confirms a signed, sealed agreement.
- Secret hygiene by design: `push_profile_db()` and `rebrand-clients.py` deep-copy then pop `relay.resend_api_key` before any DB/public write; the key lives only in gitignored `.private.json`. (One coupling: `safeguards.send_alert` reads the key from `aureon.private.json` to send alerts.)
- `compliance.py` patterns are deliberately specific (prohibition language + an email/advertising object) with a positive/negative self-test, so a site merely selling "marketing services" does not trip.
- `brand_autofill` MERGES rather than replaces: a 2026-06-12 fix preserves operator-owned keys (template, legal, tagline, cta_url) after a bare replace was clobbering custom templates every 15 min.
- E-sign produces a tamper-evident record: SHA-256 over the exact agreed bytes, a certificate panel, a locked PDF, and a second independent evidence email.
- The no-em-dash house style is enforced in three places: the AI copy prompt, a post-scrub in `onboard-pipeline`, and `contract_lib._nodash`.
- Known self-serve gaps: ICP -> niche YAML is still semi-manual, auto-DNS only works for zones a token is held for, and give-first offer fulfillment is not automated.

5.8 Client surfaces, desktop & ops

Purpose. The client-facing surfaces and operational backbone: a hosted Supabase-backed SaaS portal, an internal Tauri desktop control panel over the self-hosted Docker stack, and the

container/cron/Windows-task plumbing plus watchdog/keep-awake/hardening/tracking that keep the pipeline running hands-free.

Module	Role
saas/src/App.tsx	React Router SPA shell (public/protected/admin routes)
saas/src/lib/api.ts	All Supabase reads/writes + Edge Function calls (createSubmission , metrics, contracts, CRM)
saas/src/lib/auth.tsx	AuthProvider (password, magic-link, reset); reads user_roles.is_admin
saas/src/routes/Onboard.tsx / Sign.tsx / Status.tsx / Dashboard.tsx / Admin.tsx	Intake form, self-serve e-sign, provisioning ladder (4s poll), RLS-scoped live metrics (20s refresh), admin console
desktop/src-tauri/src/commands.rs / docker.rs / portable.rs	Tauri commands; docker compose wrapper + log streaming; one-click backup/transfer (repo + bootstrap.env + 13 volume tarballs)
desktop/frontend/src/App.tsx	Internal Tauri admin UI (Overview/Leads/Campaigns/Analytics/Domains/Warmup/Bandit/Niches/Personas/Logs/Settings)
docker/docker-compose.yml	Full \$0/mo self-hostable v4 stack (design artifact)
deploy/Dockerfile + deploy/crontab	python:3.12-slim image running cron; 1:1 mirror of the Windows LES-* tasks
scripts/watchdog.py	Hourly self-healing: stuck-runner detection, re-enable scrape tasks, refill pool, [WATCHDOG] alerts
scripts/harden-task-xml.py	Patches Task Scheduler XML (WakeToRun, RestartOnFailure, ExecutionTimeLimit)
workers/track-open.worker.js	Cloudflare Worker serving the open pixel + click redirect, PATCHing send_log via service-role key

Data flow. A client fills `Onboard.tsx`. `createSubmission` does NOT write the DB directly (public form has no anon write under RLS); it invokes the `auth-admin` Edge Function (action "submit"), which with the service role creates/links an auth user, inserts the submission, and returns a magic-link `token_hash`. The SPA immediately `verify0tp`s to establish a session, then routes to `/sign/:id`. The PC pipeline writes `contracts` + `provisioning_status`; `Sign.tsx` polls for the draft and writes the signature back; `Status.tsx` polls the ladder every 4s; `Dashboard.tsx` reads RLS-scoped metrics once live. Separately the Tauri desktop panel drives the SELF-HOSTED v4 stack over IPC against a local Postgres with a different schema. The Cloudflare Worker writes open/click timestamps straight into `send_log`.

External deps. Supabase (Auth/Postgres/Edge Functions/RLS); React 18/Vite 5/react-router 6; Tauri 2 + Rust crates (tokio, reqwest, tokio-postgres, zip); Docker + docker compose; Resend; Cloudflare Workers; Hostinger/Cloudflare DNS APIs; Windows Task Scheduler + Win32 keep-awake; ipify.org (best-effort signer IP).

Notable decisions.

- Two separate frontends with two separate backends: the public SaaS portal talks to hosted Supabase; the internal Tauri panel talks to a self-hosted compose Postgres with a DIFFERENT schema. They are not the same database.

- Onboarding deliberately bypasses anon RLS by routing the first write through the `auth-admin` Edge Function, which also returns a magic-link `token_hash` to seamlessly sign the client in.
- The Tauri frontend has an honest no-mock fallback: outside the Tauri shell, read commands return `[]` /null and the UI shows "stack not connected" rather than fake data.
- `deploy/crontab` is a 1:1 mirror of the Windows `LES-*` tasks, so the same Python runs unchanged on a VPS or laptop; only the scheduler changes.
- The Cloudflare Worker exists because Resend sets tracking flags but does not inject the pixel/rewrite links; the Worker PATCHes `send_log` via the service-role key.
- `n8n-workflows/` is the documented legacy predecessor (Google Sheets + Apollo/Anymail/Instantly/Close), now inactive.

6. End-to-End Data Flow

Follow one prospect, a small US real-estate brokerage, for the `real_estate_us` niche (client `profile_slug`).

1. **Discovery.** A scheduled driver runs `places_scrape.py`, which reads the `places:` block of `niches/real_estate_us.yaml`, queries Serper `/places` for "real estate agents in <city>", and visits the brokerage website's contact/about pages. `compliance.forbids_outreach` passes (no anti-solicitation notice). It extracts `info@brokerage.com` on the business's own registrable domain, sets `company = business name + city`.
2. **Quality + name derivation.** `name_derive` derives a first name/company where possible; the niche's `require_first_name` switch and city requirement decide admission. `_is_junk_email` rejects structural false matches.
3. **Verification.** `lead_verify.verify()` runs syntax -> junk-local -> disposable -> placeholder -> MX. Because port 25 is blocked locally, the scrape is MX-only (`method=mx_verified`, `verified=True`). The row upserts into `prospects` on conflict (`profile_slug`, `email`).
4. **Enrichment.** `context_autofill` later picks the row (null `enriched_at`), derives the website URL, runs `context_enrich` to pull product/pricing/outcomes/socials into `enriched_context`, and PATCHes `enriched_at`. `enrichment_worker` adds `industry_tags`, `geo`, and a `quality_score`; `prospect_timezone.resolve_timezone()` sets the IANA tz for send-window enforcement. The daily `smtp-pool-verify` on the VPS runs a real RCPT probe and would suppress only on a definitive 550/551/553.
5. **Enrollment.** An enrollment step creates a `runs` row (UNIQUE `sequence_id`, `prospect_id`, `status='queued'`, `next_send_at=now`).
6. **Step 1 send.** `sequence-runner.tick()` finds the due run via `idx_runs_next_send_at`. It resolves step 1's copy from `variants/sequence_steps` (no A/B inline here), calls `pick_persona_and_domain` to pick the warmed (persona, subdomain) with the most remaining quota, and `materialize_persona` binds the persona to its own `from_addr`. Gates pass (verified, not unsubscribed, required merge fields present), then `safeguards.check_all` clears the recipient-local send window and reputation/cap/rate/dedup guards. `build_payload -> render_html` produces a plain step-1 personal email (no CTA) through the client's `email_template_*`, `_inject_tracking` adds the pixel/click rewrites, Reply-To is [`client mailbox`, `info@aureonglobal.de`], and the payload POSTs to Resend

with the per-client key. `log_send` writes a `send_log` row; `advance_run` schedules step 2 at `now + delay_days` (+/-2h jitter).

7. **Follow-ups.** On later ticks, steps 2-5 reuse the run's STICKY sender (no mid-thread drift) and render the branded CTA; the runner SKIPS the tick if that subdomain is capped/paused. Steps 6-7 stay clean.
8. **Reply.** The prospect replies to `info@aureonglobal.de`. `imap-poll.py` classifies it as a reply, matches it back via In-Reply-To/References (or the recipient+subject fallback), inserts a `replies` row, marks the `send_log` row `replied`, pauses EVERY queued run for that email (`paused_replied`), and emails a `[REPLY ALERT]`.
9. **AI response.** `reply-autodraft.py` reads the new reply, passes suppression + active-prospect gates, resolves brand+persona from `run_id`, and (for positive call-intent) sends a Calendly auto-reply, or drafts a short Hormozi-voice reply via the local Claude CLI and AUTO-SENDS it threaded from the original sender (BCC `info@`), stamping `raw_headers.autosent`.
10. **Reporting.** `daily-report.py` and `client-reports.py` aggregate `send_log` / `replies` / `prospects` / `runs` (attributed by sending subdomain), render branded HTML, and email the client and `info@aureonglobal.de`. `notify_daemon.py` had already fired a Windows toast on the reply.

7. (See Section 8 for the deep deliverability treatment.)

8. Sending & Deliverability Architecture

This is the most fragile, time-constant-bound part of the system. Lead sourcing, enrichment, and rendering are bounded by compute and can be retried freely; deliverability is bounded by a real-world clock. Domain warmup takes weeks of calendar time, reputation recovers slowly, and a single misconfigured loop can burn a subdomain that took a month to warm. The design is therefore conservative by default: hard caps, calendar-derived warmup, per-recipient windows, and "suppress only on certainty."

Send path

`load_profile` merges `profiles/<slug>.json` with the gitignored `.private.json` (the Resend key) and builds `relay.from_domains[]` — a pool of independently warmed subdomains. Per send, `sequence-runner.py` (mirroring `resend-pool-send.py`) calls `pick_persona_and_domain`: filter to verified + reputation-OK subdomains with room under `daily_target_for_domain` (a snowball curve at the subdomain's `warmup_day`), then the least-used persona past cooldown on the most-room subdomain. `materialize_persona` enforces 1:1 binding. `safeguards.check_all` runs six guards; the first failure aborts only that attempt. `email_render.build_payload` builds the Resend JSON (formataddr From, one-click List-Unsubscribe, step-1 no CTA) and POSTs `api.resend.com/emails` with the send-only key. The outcome goes to `warmup-state/<slug>.resend.jsonl` and `send_log` (`sent_at` in UTC).

Persona <-> subdomain 1:1 binding

Hardcoded standard: each persona only ever sends from its own dedicated subdomain (`materialize_persona` never rebinds to a foreign sub), with Reply-To on `info@<root>`. This isolates

reputation: a problem with one persona's subdomain cannot contaminate another. `aureon.json` shows 12 subdomains for 12 personas.

Warmup curve

Warmup day is derived from `warmup.started_at` calendar age, NOT the stored `current_day`. This is a deliberate fix: an `advance_only_mode` bug had inflated `current_day` to 100+ on subdomains that were only days old (lk/dorian read ~118 vs ~4 real days), which would have opened cold subdomains at full blast. `warmup-scheduler.py` runs a daily per-subdomain snowball ramp (the `15 > 25 > 35 > 50` curve stamped by `set-warmup-curve.py`), splits warmup vs real volume, sends link-free warmup mail with jitter, and auto-pauses on reputation.

Quota model

`send_ramp` quota is per-persona-per-day TOTAL across all that persona's domains, not per (persona, domain) pair. A prior per-pair bug let AlgoAlpha send 157/day against a 36 target (2026-06-11 audit). `fetch_today_log` MUST be filtered per-profile by sending subdomain, because persona slugs (alex, casey, sam) collide across clients; an unfiltered global log silently zeroed new clients (lk-advertising saw 0 sends despite 70/day capacity).

Safeguards and send windows

`safeguards.check_all` runs six guards against `send_log`: no-solicitation backstop, recipient-LOCAL send window (Mon-Fri 08-17 via `prospect_timezone.resolve`), rolling-72h subdomain reputation, global daily cap, rate limit, and recipient+step dedup. Guard 1 (reputation) has a 20-send minimum-sample floor, and OWN_DOMAINS self-tests are excluded from reputation. A common false alarm: "sent 0 today" is usually the recipient-local window, not a bug. A subtle prior bug stored `sent_at` as naive-local + "Z", making `check_rate_limit` see future sends and block everything; `sent_at` is now stored UTC.

Outcome reconciliation

Outcomes return via a Svix-signed Resend webhook (`resend-webhook.py`) when deployed, otherwise the hourly `resend-status-reconcile.py` poller; both patch `send_log`, suppress bounced/complained, pause runs, and refresh the 7-day `warmup.reputation` read by auto-pause and Guard 1. Because the webhook is not reliably deployed, the poller and Guard 1 read `send_log` directly so auto-pause works without it.

DMARC / SPF / DKIM and provisioning

New subdomains enter via `provision_subdomain.py` / `domain_autoprovision.py`: Resend create-domain -> push DKIM + SPF + DMARC to Hostinger DNS -> poll verify -> stamp `verified_at`. `deliverability-score.py` is a preflight 0-100 scorecard over live DNS plus warmup maturity and reputation. `fix-dmarc.py` performs multi-provider DMARC repair (Cloudflare/Spaceship/Hostinger) for Gmail/Yahoo bulk-sender rules. `lk-advertising.site` and `ener-g-beratung.de` still need MANUAL DMARC.

Suppression

`suppress-bounced.py` runs cross-profile: any `send_log.bounced` flips the prospect to `verified=false` + `unsubscribed=true`. Domain auto-suppress fires only on high RATE ($\geq 50\%$, ≥ 2 bounces, ≥ 3 sent), not raw count, to avoid over-reacting to small samples. Because port 25 is blocked locally, Resend bounces are the primary dead-address signal.

Operational gotchas baked into the code

- Resend's CDN 1010-blocks the default urllib UA, so scripts spoof a browser UA; Resend's ~5 req/sec cap is self-throttled to ~4/sec.
- `save_profile` strips `resend_api_key` before any git-tracked write; separate client Resend accounts use per-profile Hostinger tokens.
- Two architectures coexist on paper: the live Resend-over-subdomains path, and the documented Postal-on-Oracle mesh (not live).

9. Multi-Tenancy & Per-Client Model

Each client is a `profiles/<slug>.json` (plus gitignored `.private.json` for secrets) and a `profiles` DB row. The profile is the per-client data model: slug (the canonical FK across every table), company, brand block (colors/fonts/legal/unsub template/template key), relay (Resend backend + `from_domains` with per-subdomain warmup), personas (1:1 bound to subdomains), rotation, warmup ramp curve, and a recipient-local `send_window`.

Isolation dimension	Mechanism
Sending identity	Per-client Resend key (in <code>.private.json</code>), per-client subdomains, persona <-> subdomain 1:1 binding
Reputation	Independent warmup state and reputation per subdomain; one subdomain's trouble cannot reach another's
Branding	Per-client <code>email_template_*.py</code> (8 today: aureon, algoalpha, lk, f2, atalsolidrocks, diraya, energ, marketing); brands without a custom template fall through to a generic brand-driven layout; <code>brand_autofill</code> mines and merges visual identity
Copy	Per-client <code>variants.json</code> -> <code>variants / sequence_steps</code>
Reporting	<code>client-reports.py</code> rebrands with the client's real site logo+colors and scopes to that <code>profile_slug</code> ; reports go to client + info@
Quota / logs	<code>fetch_today_log</code> filtered per-profile by sending subdomain (persona slugs collide across clients)
Database access	RLS: <code>owns_slug()</code> scopes sequences/replies/runs; <code>client_id</code> ownership scopes invoices/sales/contracts/provisioning_status; <code>is_admin()</code> for the operator

The persona-subdomain 1:1 binding and the per-profile log filtering are the two invariants that make true multi-tenancy work on a single shared Supabase project. Both were the subject of real incidents (AlgoAlpha over-sending, lk-advertising zeroed) before being hardened.

10. Client Surfaces & Productization

The agency is being turned into a self-serve SaaS. The surfaces:

- **SaaS portal** (`saas/` , **Vite + React + TS**). Public onboarding form, self-serve e-sign, a live provisioning-status ladder, an RLS-scoped per-client campaign dashboard (leads, sends, delivered %, replies, sales, invoices, contract), and an admin console (invite clients, review signed-contract evidence, add invoices/sales). It talks only to hosted Supabase and holds only the anon key in `saas/.env` ; all privileged work runs server-side via the PC pipeline and the `auth-admin` Edge Function.
- **Onboarding**. A submission's `raw_answers` drives `onboard-pipeline.py` , which is hard-gated on a sealed contract before any provisioning.
- **Contracts / e-sign**. `contract-sign.py` + `contract_lib.py` produce a tamper-evident record: SHA-256 over the agreed bytes, an electronic-signature certificate panel (IP, user-agent, hash), a Playwright-rendered locked PDF, and dual evidence emails. The contract clauses are inherited verbatim from the Diraya base template, with `verify_clean()` blocking any base-template token leak or em dash.
- **Reports**. `daily-report.py` is the engine; `client-reports.py` , `dorian-report.py` , and `volume-report.py` reuse or mirror it so all reports stay visually and logically consistent.
- **Desktop control panel** (`desktop/` , **Tauri 2**). The operator's internal tool over the self-hosted v4 Docker stack: lifecycle controls, streamed logs, KPI dashboards from a local Postgres, and one-click portable backup/transfer (repo + `bootstrap.env` + 13 volume tarballs). It is a different frontend over a different backend than the SaaS portal.

11. Deployment, Runtime & Operations

Where the code runs

Environment	What runs	Scheduler
Operator PC (Windows)	The live Python spine (<code>sequences/</code> , <code>scripts/</code>), <code>notify_daemon</code> , the Tauri desktop panel	Windows Task Scheduler <code>LES-*</code> tasks
VPS (port-25-open)	Real SMTP probing (<code>smtp-pool-verify</code> , <code>reacher_verify</code>) and the optional <code>deploy/</code> cron image	Linux cron (<code>deploy/crontab</code> , 1:1 mirror of <code>LES-*</code>)
Supabase (hosted)	Postgres + PostgREST + Realtime + Auth + the <code>auth-admin</code> Edge Function	n/a
Cloudflare	<code>track-open.worker.js</code> open/click tracker	edge
Docker (design / services)	<code>docker/enricher</code> , <code>docker/bandit-scorer</code> , <code>docker/sourcing</code> services; the full v4 compose is a design artifact	docker compose

Scheduled tasks

The `LES-*` Windows tasks tick the runner, warmup, scrapers, enrichment workers, verification, reply handling, and reports. `deploy/crontab` mirrors them 1:1 so the same code runs unchanged on a VPS. `harden-task-xml.py` patches the exported task XML to set `WakeToRun`, `StartWhenAvailable`, battery flags, `RestartOnFailure`, and `ExecutionTimeLimit`.

Self-healing and uptime

`watchdog.py` runs hourly: it detects a stuck `sequence-runner`, re-enables protected lead-scrape tasks, refills an exhausted prospect pool (spawning `pool-monitor.py`, which keeps a 2x buffer per profile), checks keep-awake, and emails `[WATCHDOG]` alerts via Resend only when it actually acts. `keep-awake.py` uses `Win32 SetThreadExecutionState` to stop the machine sleeping. `safeguard-domain-check.py` runs a daily 07:30 check that pool domains stay verified with healthy DKIM/SPF/DMARC.

State export / import

`desktop/src-tauri/src/portable.rs` (and the headless `export-state.ps1`) zip the repo + `bootstrap.env` + 13 named Docker volume tarballs with a `META.json` for one-click transfer to another PC; import restores them. The bundle deliberately warns that `bootstrap.env` contains secrets and should be encrypted before sharing.

12. Security & Compliance

Area	Posture
RLS	Auth-scoped after migration 007 (<code>is_admin()</code> / <code>owns_slug()</code> SECURITY DEFINER helpers, per-owner policies). <code>prospects</code> has a separate token-scoped policy: anon INSERT + token-scoped UPDATE for unsubscribe, no SELECT/DELETE
Auth	Supabase Auth (password, magic-link, reset) in the SaaS portal; admin flag from <code>user_roles.is_admin</code> ; the public form bypasses anon RLS via the service-role <code>auth-admin</code> Edge Function
Key separation	Backend scripts use <code>SUPABASE_SERVICE_KEY</code> (bypasses RLS); public pages use <code>SUPABASE_ANON_KEY</code> . Resend full key and DNS tokens never reach the browser
Secret hygiene	Per-profile <code><slug>.private.json</code> (gitignored) holds the Resend key; <code>save_profile / push_profile_db / rebrand-clients.py</code> strip it before any git/DB/public write. <code>saas/.env</code> holds only the Supabase URL + anon key
Unsubscribe	One-click <code>List-Unsubscribe</code> header on every send; brand-matched GitHub-Pages unsub pages call the <code>unsubscribe_by_token</code> RPC; unsubscribe is the token-scoped anon UPDATE on <code>prospects</code>
No-solicitation	<code>compliance.forbids_outreach</code> aborts scraping any page with EN no-solicitation or German Impressum anti-Werbung text and flags <code>prospects.custom_fields.no_solicitation</code> ; <code>safeguards.check_all</code> re-checks that flag at send time as a backstop
Legal-source stance	Published-contact sourcing only (team pages, Places, Impressum, public creator About blobs); never private/credentialed data
Hard suppression	<code>laso.finance</code> (an active legal case) plus own-brand domains are never alerted, paused, or auto-answered, enforced in both <code>imap-poll</code> and the draft scripts
LLM sandboxing	The reply-drafting Claude CLI runs with a full system-prompt replacement, all tools disabled, and an empty temp cwd, so it cannot inspect the workspace
Contract integrity	SHA-256 over the agreed bytes, signer IP/user-agent capture, a locked PDF, and a second independent evidence email

Area	Posture
Edge CSP	<code>tauri.conf.json</code> ships a tight CSP (script-src 'self', scoped connect-src)

Two known security-relevant gaps from the maps: roughly 87 anon-key call sites still exist (the recommended `service_key` or `anon_key` pattern is not yet applied everywhere), and `safeguards.send_alert` reads the Resend key directly from `aureon.private.json` on disk to send alerts.

13. Current State vs. Target SaaS

Capability	As-is (live)	To-be (target SaaS)
Tenancy	~10 client brands on one shared Supabase project, operator-run	Self-serve multi-tenant signup
Onboarding	SaaS form + PC pipeline; ICP -> niche YAML still semi-manual	Fully automated ICP -> sourcing with no operator step
Contracts	E-sign live (migrations 005+006), hard-gates provisioning	Unchanged; this is solved
Auth/RLS	Auth-scoped (007); ~87 anon-key call sites remain	All server calls on the service key; anon strictly token-scoped
Domains/DNS	Auto-DNS only where a zone token is held; some manual DMARC	Auto-DNS for any client zone
Sending	Resend over per-client subdomains, live	Unchanged (the documented Postal/Oracle mesh stays design-only)
Replies	Auto-send live; meeting-followup approve-first	Unchanged
Reporting	Branded per-client reports live	Unchanged
Lead sourcing v4	<code>docker/sourcing</code> framework not wired in; most engines stubs	Wired-in universal sourcing if needed
Deliverability	Conservative warmup + safeguards, healthy at current scale	Same controls, more domains
Desktop panel	Tauri over self-hosted compose Postgres (different schema)	Unchanged operator tool

The build constraint is effectively solved: a working multi-client cold-email platform was built solo with AI in about a month. The binding constraints are now external and largely outside the codebase:

- **Distribution.** Getting clients in the door, not shipping more features.
- **Deliverability reputation.** A real-world time constant. Warmup takes weeks of calendar time per domain; no amount of code shortens it.
- **Payment collection.** Invoices/sales tables exist (008) but collection is not automated.
- **Single-operator support load.** One person handles every client's replies, exceptions, and DNS edge cases; this caps client count more than compute does.

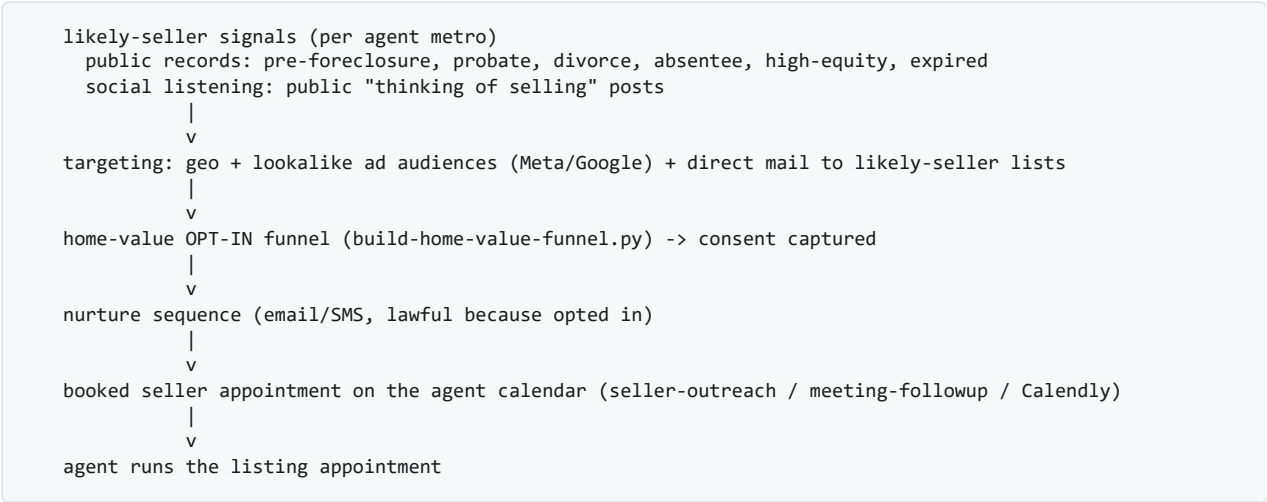
14. Planned: Intent-Signal Enrichment Layer (Seller-Appointment Engine)

Status: specified, not yet built. Full blueprint in `docs/INTENT_SIGNAL_LAYER_SPEC.md`; this section folds that spec into the architecture.

Objective

The product's B2C job is to produce booked SELLER APPOINTMENTS for each signed real-estate agent in that agent's local area. The B2B cold-email layer signs the agents; this layer fills their calendar with listing appointments. It is the front-end that feeds qualified likely-sellers into the seller-appointment machine that already partly exists (`build-home-value-funnel.py` , `seller-outreach.py` , `meeting-followup.py` , the sequence engine, `referral-lists/metros/`).

Consent-gated seller-appointment flow



The homeowner is emailed only after opting into the funnel. Cold signals drive targeting and the funnel; they never cold-email the homeowner. Consumer cold email is illegal in the EU and deliverability-poison in the US, so it is structurally excluded.

Engine design (config-driven, AI-constrained)

The standardization requirement: the AI never freelances. Signals live in YAML packs; a Python orchestrator expands each signal into a fixed instruction and the AI fills a rigid evidence-required JSON schema (`{found, evidence_url, evidence_snippet, event_date, confidence}`). Deterministic scoring produces a weighted, recency-decayed score per prospect written to the existing intent layer (`profiles.lead_intent` / migration 004). Same inputs always yield the same instructions and the same score; no qualifying public evidence means no signal.

Module	Role
<code>niches/signals/<pack>.yaml</code>	Signal pack: signals, sources, weights, recency, channel routing, jurisdiction, tone
<code>sequences/signal_pack_lib.py</code>	Loader + schema validation + jurisdiction / protected-class / allowlist enforcement
<code>sequences/intent_signals.py</code>	Per-prospect orchestrator: render queries, dispatch to Serper, validate evidence

Module	Role
sequences/intent_score.py	Deterministic weighted + recency-decayed scoring

Real-estate signal pack (priority vertical, US)

Public-record distress signals (route to legal direct mail) plus social-listening public-intent signals (voluntary public requests, route to funnel / agent follow-up):

Signal	Source	Weight	Channel
Pre-foreclosure / NOD / lis pendens	County recorder	0.95	direct_mail
Sheriff / auction notice	Public legal notices	0.95	direct_mail
Divorce filing	County court	0.9	direct_mail
Probate / inherited	Probate court	0.85	direct_mail
Bankruptcy	PACER	0.75	direct_mail
Tax lien / delinquency	County tax rolls	0.7	direct_mail
Expired / withdrawn listing	MLS-adjacent	0.6	optin_funnel
Absentee / vacant	Mailing != property addr	0.55	direct_mail
"Recommend a realtor" (public post)	Reddit / Nextdoor / X	0.9	reply or DM -> funnel
"Relocating, need to sell" (public post)	Reddit / X / forums	0.85	funnel / agent follow-up
"Just inherited, what now" (public post)	Reddit / forums	0.8	optin_funnel

Jurisdiction split

Client jurisdiction	Pack	Signals
US (real estate)	us_real_estate_distress	Public-record distress + public-intent social listening
EU (ENER-G, f2, etc.)	eu_b2b	Company-level only: hiring, funding, expansion, tech-stack, leadership, reviews. No individual life-event data (GDPR)

Guardrails (enforced in code)

1. **Jurisdiction gate.** A pack's jurisdiction must match the client; US-distress packs never run on EU clients.
2. **Source allowlist + social ToS.** Only allowlisted public sources, each tagged with an access method; public/API/indexed access (the `linkedin_via_google` pattern), never mass scraping of locked platforms.
3. **Voluntary-solicitation only for social.** Social focus is people who publicly ask for the service; involuntary private misfortune dug from a feed is out of scope (GDPR-illegal for EU, ad-platform sensitive-category banned, brand-damaging).
4. **Fair Housing.** Event and financial signals only, never protected classes.

5. **No consumer cold email.** B2C signals route to `optin_funnel / ad_audience / direct_mail / prioritize_existing`; consumer email only to recorded opt-ins. Cold email stays the B2B tool for signing agents.
 6. **Humane tone.** A mandatory per-pack tone block; distress is never weaponized in outreach.
-

15. Appendix — Module / Directory Index

sequences/ (the live Python spine)

- Runtime: `sequence-runner.py`, `resend-pool-send.py`, `warmup-scheduler.py`, `profile_lib.py`, `safeguards.py`, `email_render.py`
- Templates: `email_template_aureon.py`, `email_template_diraya.py`, `email_template_energ.py` (+ `algoalpha`, `lk`, `f2`, `atalsolidrocks`, `mark-eting`)
- Sourcing: `lead_scrape.py`, `places_scrape.py`, `seed_discover.py`, `impressum_scrape.py`, `auto_source_atal.py`, `youtube_scraper.py`, `youtube_worker.py`, `tradingview_scrape.py`, `social_scrape.py`, `crypto_projects_scrape.py`, `search_dispatch_worker.py`, `name_derive.py`
- Enrichment/verify: `context_enrich.py`, `context_autofill.py`, `enrichment_worker.py`, `personal_hook_worker.py`, `prospect_timezone.py`, `lead_verify.py`, `reverify-unknowns.py`, `reverify-pool.py`
- Provisioning/deliverability: `provision_subdomain.py`, `domain_autoprovision.py`, `deliverability-score.py`, `resend-status-reconcile.py`, `resend-webhook.py`, `hostinger-smtp-send.py`, `safeguard-domain-check.py`
- Onboarding/legal/brand: `onboard-pipeline.py`, `contract-sign.py`, `contract_lib.py`, `compliance.py`, `brand_extract.py`, `brand_autofill.py`
- Replies/notify: `imap-poll.py`, `reply-autodraft.py`, `seller-outreach.py`, `meeting-followup.py`, `notify_daemon.py`
- Data/sync: `supabase_sync.py`, `queue_lib.py`
- Copy: `<slug>-default/variants.json`
- Previews/test: `render_email_preview.py`, `render_previews.py`, `send-sequence.py`

scripts/ (ops, reporting, tooling)

- Reporting: `daily-report.py`, `client-reports.py`, `dorian-report.py`, `volume-report.py`, `daily-status.py`, `_ops-live-stats.py`, `_reply-correlation-analysis.py`
- Verification: `smtp-pool-verify.py`, `smtp-verify.py`, `reacher_verify.py`
- Sequence wiring/QA: `wire-sequence-steps.py`, `render-all-steps.py`
- Deliverability ops: `suppress-bounced.py`, `fix-dmarc.py`, `pool-monitor.py`, `set-warmup-curve.py`
- Onboarding ops: `site_style.py`, `build-unsub-pages.py`, `scaffold-niche.py`, `rebrand-clients.py`
- System ops: `watchdog.py`, `keep-awake.py`, `harden-task-xml.py`
- Autonomous research: `research-dispatcher.py`

docker/ (services + v4 design)

- `enricher/` (FastAPI deep enrichment + modules `website/social/infra/techstack/news/signals`)

- `bandit-scorer/bandit_scorer.py` (Thompson bandit + lead scoring)
- `sourcing/` (v4 universal sourcing FastAPI + engine registry; not wired into the live pipeline)
- `route-picker/route_picker.py` (Postal-era route picker; design)
- `litellm/config.yaml` (OpenAI/Anthropic -> Ollama gateway)
- `docker-compose.yml` (full \$0/mo v4 stack; design artifact)

`supabase/`

- `schema.sql`; `migration_003_scale_pipeline.sql` through `migration_008_crm.sql`; `config.toml`
- (RLS hotfixes live in `docs/SUPABASE_RLS_FIX.sql` and `_V2.sql`)

`saas/` (client portal, Vite + React + TS)

- `src/App.tsx`, `src/lib/api.ts`, `src/lib/auth.tsx`
- `src/routes/Onboard.tsx`, `Sign.tsx`, `Status.tsx`, `Dashboard.tsx`, `Admin.tsx`

`desktop/` (Tauri 2 control panel)

- `src-tauri/src/commands.rs`, `docker.rs`, `portable.rs`
- `frontend/src/App.tsx` (+
Overview/Leads/Campaigns/Analytics/Domains/Warmup/Bandit/Niches/Personas/Logs/Settings)

`deploy/` and `workers/`

- `deploy/Dockerfile`, `deploy/crontab`, `deploy/docker-compose.yml` (VPS cron mirror)
- `workers/track-open.worker.js` (Cloudflare open/click tracker)

Top-level docs

- Ground truth: `INSTALL.md`, the `sequences/` `code`, `supabase/`, `profiles/`
- Vision/roadmap (not the running system): `README.md`, `ARCHITECTURE.md`, `INNOVATIONS.md`, `SOURCING_AND_NICHES.md`
- Runbooks/specs: `docs/CLIENT_ONBOARDING.md`, `docs/LEGAL_LEAD_SOURCING_UPGRADE.md`, `profiles/_schema.md`, `TODO-2026-06-12.md`